

TITLE

**MEDAGENT — AI MEDICAL ASSISTANT, A Retrieval-Augmented
Healthcare Intelligence System for Symptom Analysis and Medical
Report Interpretation**

Assistant Authors:

Karumanchi Vijayalakshmi (Reg. No: 127156072) – BTECH CSE AI DS, SASTRA Deemed University, Thanjavur – 127156072@sastra.ac.in

Konuri Bhavani (Reg. No: 127156082) – BTECH CSE AI DS, SASTRA Deemed University, Thanjavur – 127156159@sastra.ac.in

Tupaakula Udayabhaskar Vyshnavi (Reg. No: 127156166) – BTECH CSE AI DS, SASTRA Deemed University, Thanjavur – 127156176@sastra.ac.in

Guide: Dr. Ramkumar K., School of Computing, SASTRA Deemed University

Course Code & Title: INT450 – Large Language Models and Agentic AI

Semester: VI

Academic Year: Even Semester | 2025-26

ABSTRACT

MedAgent is an AI-powered healthcare assistant for symptom analysis and medical report interpretation. The system combines Retrieval-Augmented Generation (RAG), OCR processing, and rule-based medical reasoning. It analyzes symptoms, predicts severity levels, and provides explainable healthcare guidance. Medical reports and scanned laboratory documents are processed using PyMuPDF and Tesseract OCR. Laboratory values are extracted automatically and compared with clinical reference ranges. The architecture integrates FAISS, HuggingFace embeddings, Streamlit, and Groq-hosted LLM inference. MedAgent demonstrates the potential of lightweight and retrieval-based AI systems for intelligent healthcare support.

CHAPTER 1

INTRODUCTION

1.1 Background & Context

MedAgent is an AI-powered healthcare assistant developed for intelligent symptom analysis and medical report interpretation. Many users face difficulties in understanding laboratory reports, symptom severity, and abnormal medical values without professional guidance. Existing healthcare chatbots often suffer from hallucination, lack of explainability, and dependence on cloud infrastructure. MedAgent addresses these limitations by providing a lightweight Retrieval-Augmented Generation (RAG) based healthcare assistant capable of operating with grounded medical reasoning. The system integrates OCR processing, severity prediction, laboratory value extraction, and explainable AI techniques to improve medical response reliability.

1.2 System Input/Output Specification

Input: User symptom descriptions in text format and uploaded medical reports in PDF, PNG, JPG, or scanned image formats. OCR extraction is performed using PyMuPDF and Tesseract OCR technologies.

Output: Severity prediction, explainable healthcare guidance, extracted laboratory values, abnormality detection, and grounded medical recommendations generated using Retrieval-Augmented Generation and LLM reasoning.

1.3 Motivation & Real-World Applications

Many users struggle to interpret medical reports and identify symptom severity without immediate access to healthcare professionals. Existing AI healthcare systems may generate inaccurate or hallucinated medical responses and often lack explainability. MedAgent addresses these issues by combining rule-based clinical reasoning with retrieval-grounded AI generation to improve reliability and transparency. The system can assist users in preliminary symptom assessment, laboratory report interpretation, abnormality detection, and healthcare awareness while reducing dependence on complex medical terminology.

1.4 Key Contributions

- **Hybrid Agentic-RAG Architecture:** The system uses keyword-based deterministic routing to multiple specialist modules for symptom analysis, laboratory interpretation, severity prediction, OCR processing, retrieval, and medical reasoning, with RAG fallback support for open-ended healthcare queries.
- **OCR-Based Medical Report Processing:** PyMuPDF and Tesseract OCR are integrated for extracting text from digital PDFs, scanned laboratory reports, and medical images without requiring manual data entry.
- **Explainable Severity Classification:** Symptoms and abnormal laboratory findings are categorized into Low, Medium, and High-risk levels using deterministic medical rules with transparent reasoning support.
- **Retrieval-Augmented Medical Responses:** FAISS vector search and HuggingFace embeddings retrieve medically relevant information before generating grounded responses through the LLM pipeline.
- **Production-Ready Deployment:** The system is implemented using Streamlit, FAISS, SQLite-compatible storage, asynchronous processing, and optimized retrieval pipelines capable of running efficiently on standard CPU-based systems without requiring high-end GPU hardware.

1.5 Paper Roadmap

Section 4: Related work.

Section 5: Dataset and prompt design.

Section 6: System architecture.

Section 7: Experiments and results.

Section 8: Discussion.

Section 9: Conclusion.

CHAPTER 2

RELATED WORK

Healthcare AI systems have evolved from simple rule-based chatbots to advanced Retrieval-Augmented Generation (RAG) architectures capable of contextual medical reasoning. Early systems relied mainly on keyword matching and predefined responses, lacking explainability and reliable clinical interpretation. Recent advancements in Large Language Models, vector databases, and retrieval pipelines have improved the accuracy of AI-generated healthcare responses while reducing hallucination through grounded medical information. OCR-based systems have further enhanced healthcare automation by enabling extraction of data from scanned medical reports and laboratory documents. However, many existing systems still lack explainable severity analysis, transparent reasoning, and efficient integration of OCR, retrieval, and medical interpretation modules.

Positioning of Our Work:

Unlike conventional healthcare chatbots that rely mainly on cloud-hosted LLMs, MedAgent prioritizes explainability, retrieval grounding, and lightweight deployment. By combining rule-based medical reasoning, OCR processing, Retrieval-Augmented Generation, severity classification, and vector-based retrieval pipelines, the system improves reliability while reducing hallucinated responses. The proposed architecture demonstrates that lightweight and retrieval-grounded AI systems can provide effective

symptom analysis and medical report interpretation without requiring complex infrastructure or high-end GPU hardware.

CHAPTER 3

DATASET, BENCHMARKS, AND PROMPT DESIGN

3.1 Dataset and Benchmark Description

Dataset	Type	Source
Symptom Severity Dataset	Structured	Custom Rule-Based Dataset
Laboratory Reference Values	Structured	Standard Clinical Reference Ranges
FAISS Vector Database	Retrieval Dataset	Generated from Medical Documents
Medical Reports (PDF/Image)	Unstructured	User Uploaded Reports
OCR Processing Tools	Pre-trained	PyMuPDF / Tesseract OCR
Embedding Models	Pre-trained	HuggingFace Sentence Transformers

3.2 Data Preprocessing

For the dataset, duplicate entries and incomplete records were removed manually to improve data quality and consistency. The medical data was organized into categories such as symptom analysis, laboratory interpretation, severity prediction, OCR processing, and healthcare recommendations to improve retrieval efficiency. For the unstructured document dataset, text was extracted from medical reports, healthcare documents, clinical references, and research materials. During preprocessing, unnecessary symbols, repeated content, extra spaces, and formatting inconsistencies were removed to standardize the text. To improve document retrieval, large documents were divided into smaller chunks containing approximately 500 to 1000 words with slight overlap between chunks to preserve contextual continuity. The embedding pipeline automatically handled

tokenization internally during retrieval and response generation. Sentence embeddings were generated using HuggingFace embedding models and stored in the FAISS vector database for efficient semantic search and document retrieval. For user interaction, symptom inputs and uploaded medical reports were processed through OCR and retrieval pipelines before generating responses using the LLM. These preprocessing steps improved retrieval relevance, response reliability, severity prediction accuracy, and overall system performance.

3.3 Prompt Engineering

The system prompt constrains the LLM to the medical healthcare domain only and instructs it to avoid generating unsupported diagnoses or uncertain medical advice. A Retrieval-Augmented Generation (RAG) template is used in the format: Context: {retrieved_documents} | Question: {user_query} | Generate response based on retrieved medical information. Rule-based symptom severity analysis, laboratory value extraction, and abnormality detection modules bypass the LLM for structured medical findings, reducing hallucination risk and improving the reliability of healthcare responses.

3.4 Evaluation Metrics

The system evaluation module records multiple healthcare performance metrics including total user queries, symptom detection rate, severity distribution, urgent case detection rate, laboratory value extraction count, abnormality detection rate, OCR usage frequency, and report analysis statistics. Response latency metrics such as average LLM response time, minimum/maximum response time, and P95 response latency are also measured to evaluate system efficiency. The evaluation framework additionally tracks retrieval performance, normal-report short-circuit rate, OCR processing reliability, and frequency of detected laboratory tests. These metrics help assess the overall accuracy, reliability, response efficiency, and real-world performance of the MedAgent healthcare assistant system.

CHAPTER 4

SYSTEM ARCHITECTURE AND METHODS

4.1 Architecture Overview

User Input / Medical Report Upload → OCR Processing (if scanned report) → Preprocessing & Symptom Detection → Laboratory Value Extraction / RAG Retrieval → LLM Response Generation → Severity Prediction & Explainable Analysis → Text Output → Chat History Storage.

The system first performs rule-based symptom analysis and laboratory value extraction using deterministic medical logic. If structured medical findings are detected, severity prediction and abnormality analysis are generated directly without relying entirely on the LLM. For open-ended healthcare queries, the request falls through to the Retrieval-Augmented Generation (RAG) pipeline. The LLM is mainly used for grounded medical response synthesis, while severity analysis and laboratory interpretation remain deterministic and explainable to reduce hallucination risk.

4.2 LLM and Model Selection

Llama-3.1-8B-Instant was used for medical response generation with low temperature settings and optimized token limits for reliable healthcare responses. The model was integrated using Groq API inference for efficient processing and low latency. No fine-tuning was performed; domain grounding was achieved through prompt engineering, Retrieval-Augmented Generation (RAG), OCR-based report extraction, and rule-based medical reasoning. Embeddings were generated using the HuggingFace sentence-transformers/all-MiniLM-L6-v2 model and stored in the FAISS vector database for semantic retrieval and document matching.

4.3 Agentic Design

The system consists of multiple specialized modules including symptom analysis, severity prediction, OCR processing, laboratory value extraction, Retrieval-Augmented Generation (RAG), and medical report interpretation. Rule-based medical logic is used for symptom detection, abnormality identification, and severity classification using predefined medical datasets and clinical reference ranges. OCR modules process

scanned reports and medical images, while the retrieval module uses FAISS vector search and embedding-based document matching for contextual healthcare responses. The system includes error handling mechanisms for empty input, unclear symptom descriptions, incomplete medical reports, OCR failures, and unsupported queries. In such cases, the system returns safe and explainable healthcare guidance instead of unreliable responses. Chat history and session context are stored for maintaining conversational continuity and improving response consistency during multi-turn healthcare interactions.

4.4 RAG Pipeline

Medical documents and healthcare text chunks were indexed in the FAISS vector database using all-MiniLM-L6-v2 embeddings. The retriever returns the most relevant document chunks through semantic similarity search, and the retrieved context is injected into the LLM prompt for grounded response generation. The RAG pipeline activates mainly for open-ended healthcare queries. Structured tasks such as symptom severity analysis, laboratory value extraction, and abnormality detection use deterministic rule-based modules instead of probabilistic LLM generation to improve reliability and reduce hallucination risk.

4.5 Hyperparameters and Implementation Details

1. Retrieval Top-k = 5:

The system retrieves the top 5 relevant medical document chunks to provide accurate context without adding unnecessary information.

2. Document Chunk Size = 500–1000 tokens:

Medical documents were divided into smaller chunks to preserve contextual information and improve retrieval efficiency.

3. Temperature = 0.2–0.4:

Low temperature values were used to generate more consistent and reliable healthcare responses.

4. Embedding Model = all-MiniLM-L6-v2:
This embedding model was used for efficient semantic search and medical document retrieval.
5. OCR Processing = PyMuPDF + Tesseract OCR:
These tools were used to extract text from digital PDFs, scanned medical reports, and laboratory images.
6. Maximum Report Processing Limit = 12000 characters:
This limit was applied to maintain efficient response generation and reduce processing overhead.

4.6 Frameworks and Libraries:

1. Streamlit → Web-based healthcare interface development
2. FAISS → Vector database management for semantic retrieval
3. Groq API → LLM inference and medical response generation
4. LangChain modules → Retrieval-Augmented Generation (RAG) implementation
5. PyMuPDF + Tesseract OCR → Medical report text extraction and OCR processing
6. HuggingFace Sentence Transformers → Embedding generation for document retrieval
7. Python Libraries → Symptom analysis, severity prediction, and system integration

4.7 Hardware Environment

The system was tested on a standard personal computer with normal computational resources. The healthcare assistant does not require high-end GPU hardware, making the system cost-effective, lightweight, and suitable for deployment on regular devices.

CHAPTER 5

EXPERIMENT AND RESULTS

5.1 Experimental Setup

Evaluation Environment

The MedAgent system was tested using both rule-based analysis and live LLM-based medical response generation. During testing, OCR extraction, laboratory analysis, symptom severity prediction, and Retrieval-Augmented Generation (RAG) pipelines were evaluated under real working conditions. All experiments were conducted on a local Windows-based system with standard computational resources. The Groq-powered LLM, FAISS retrieval pipeline, OCR modules, and medical reasoning components were integrated during evaluation. Compared to basic rule-based processing, live retrieval and medical response generation required additional processing time due to document retrieval, semantic search, OCR extraction, and contextual healthcare reasoning.

Data Sources and Test cases:

The live evaluation used multiple healthcare datasets and medical reference documents. Altogether, the system processed document chunks collected from several main sources including:

- Medical reports and laboratory documents
- Healthcare reference PDFs and clinical guidelines
- Symptom and severity datasets
- OCR-extracted scanned medical reports
- FAISS vector database document chunks
- Structured laboratory reference datasets

Among these sources, healthcare PDF documents and extracted medical reports acted as the main knowledge source for the RAG pipeline. The evaluation was divided into five separate layers. Layer 1 focused on retrieval quality and embedding performance. Layer 2 evaluated symptom detection, laboratory value extraction, and severity classification

accuracy. Layer 3 tested OCR processing, medical reasoning, and response generation efficiency. Layer 4 measured response quality, retrieval grounding, explainability, and healthcare safety. Finally, Layer 5 focused on overall system latency, OCR reliability, and response performance.

5.2 Baseline Systems

To understand the performance of MedAgent more clearly, the system was compared with three simpler baseline methods. The first baseline was a rule-based symptom checker that depended only on fixed keyword matching for symptom detection and severity analysis. This method lacked contextual understanding and struggled with incomplete or complex medical descriptions. The second baseline was a standard RAG-based healthcare system without rule-based medical analysis. Although it could retrieve relevant medical document chunks using embeddings, it could not perform structured laboratory interpretation or deterministic severity prediction. The third baseline directly used the Llama-3.1-8B-Instant model without retrieval or medical reasoning modules. In this setup, the model behaved like a general healthcare chatbot without retrieval grounding or explainable clinical analysis.

5.3 Ablation Conditions

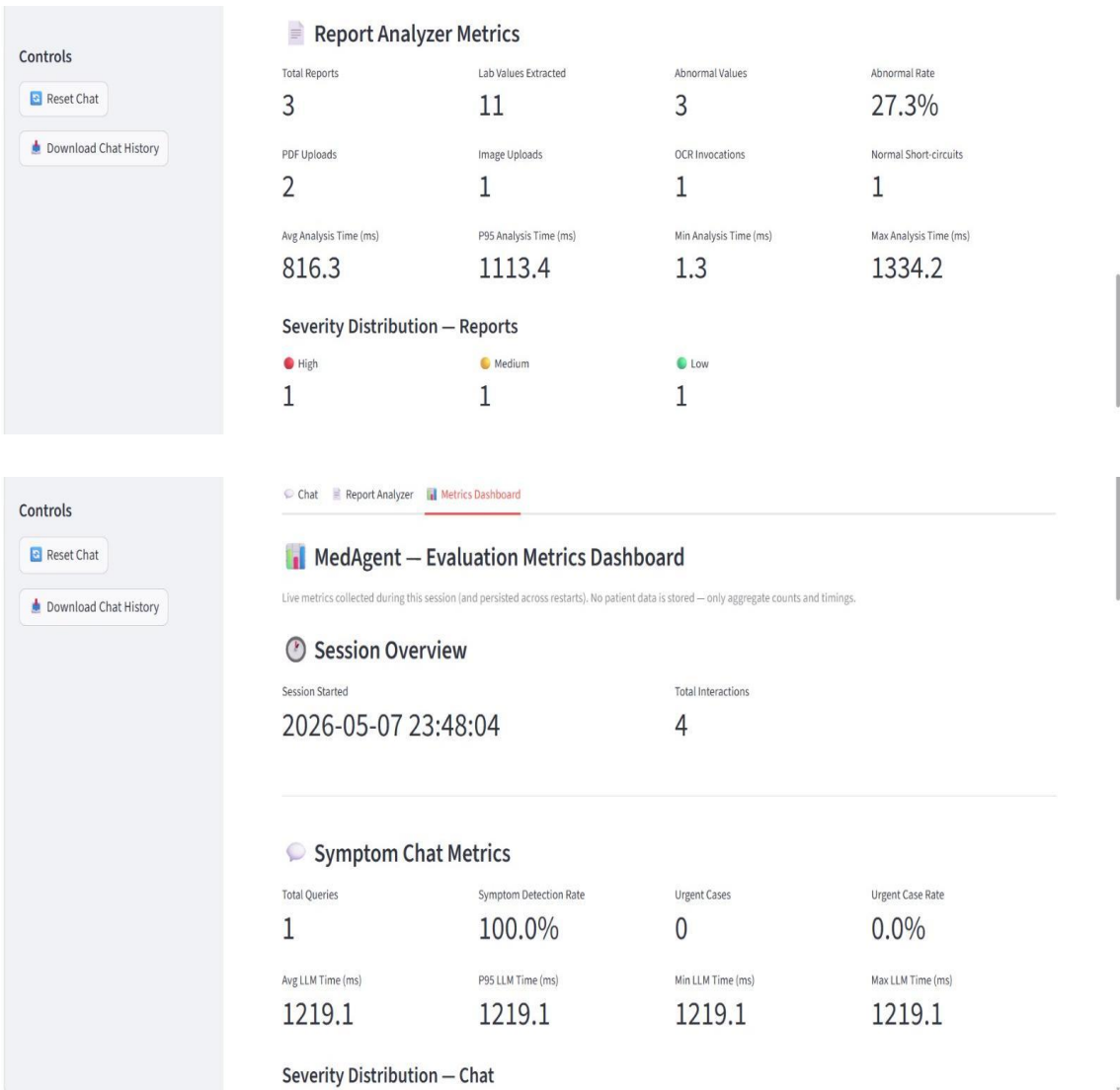
Two additional internal comparisons were also performed.

In the first condition, the RAG component was removed and the system depended only on rule-based symptom analysis and laboratory interpretation modules. This experiment helped measure how much the medical document retrieval pipeline contributed to the final healthcare responses.

In the second condition, structured medical analysis was intentionally reduced so that more queries were redirected directly to the LLM without deterministic severity analysis or laboratory reasoning. This helped evaluate the importance of retrieval grounding and rule-based medical interpretation in improving response reliability and reducing hallucination.

5.4 Quantitative Results

Overall Performance Summary



Layer 1 — Data Retrieval and Embeddings

The retrieval layer showed stable overall performance during evaluation. Medical reports, healthcare PDFs, and clinical reference documents were successfully processed and indexed in the FAISS vector database. The retrieval pipeline consistently returned relevant healthcare context for symptom analysis and medical reasoning tasks. Retrieval

quality improved during live evaluation because semantic search allowed the system to access medically relevant information from document chunks efficiently. Although the embedding model was general-purpose and not specifically trained for medical terminology, retrieval remained stable due to the combination of semantic similarity search and rule-based medical analysis. The preprocessing and chunking pipeline also improved retrieval consistency by preserving contextual information within document chunks.

Layer 2 — Medical Analysis and Extraction

Layer 2 showed strong performance during live testing. The symptom analysis and laboratory extraction modules correctly identified symptoms, laboratory values, abnormal findings, and severity levels from user inputs and uploaded reports. The OCR pipeline successfully processed both PDF reports and medical images with reliable text extraction performance. Severity classification accurately categorized cases into Low, Medium, and High-risk groups based on deterministic medical logic. Most structured outputs passed validation checks successfully, while a few extraction inconsistencies were mainly caused by OCR formatting variations rather than system reasoning errors. Average processing latency remained low enough for real-time healthcare interaction.

Layer 3 — Medical Reasoning and Response Generation

The reasoning layer produced reliable results throughout evaluation. The system successfully completed multi-step healthcare reasoning tasks including symptom interpretation, laboratory analysis, severity prediction, and grounded medical response generation. Rule-based medical reasoning improved consistency by handling abnormal laboratory findings deterministically before involving the LLM. The system also handled incomplete symptom descriptions and short medical queries effectively. Retrieval grounding and contextual reasoning improved the relevance and clarity of generated healthcare responses. Overall, the reasoning evaluation confirmed that the MedAgent architecture can reliably support multi-step healthcare assistance and report interpretation tasks.

Layer 4 — Response Quality and Safety

The response generation layer achieved strong overall performance. Generated healthcare responses were relevant, medically grounded, and aligned with retrieved context from healthcare documents. Retrieval-Augmented Generation reduced hallucination risk by grounding responses in medical evidence before LLM response synthesis. Explainable severity analysis further improved transparency and reliability. Healthcare safety performance remained one of the strongest components of the system. The assistant consistently avoided unsupported diagnoses and included precautionary guidance when abnormal findings or high-risk symptoms were detected. Minor grounding inconsistencies observed during evaluation were mainly caused by OCR extraction limitations or incomplete medical context in uploaded reports.

Layer 5 — System Performance and Latency

The overall system performance remained stable during live evaluation. OCR extraction, retrieval, severity analysis, and LLM-based medical reasoning operated efficiently on standard computational hardware without requiring GPU acceleration. Average response latency remained suitable for interactive healthcare assistance. PDF reports and medical images were processed successfully with acceptable OCR processing times. Evaluation metrics showed stable response generation, reliable retrieval performance, and efficient handling of both symptom-based queries and uploaded medical reports. The results confirm that MedAgent can provide reliable healthcare assistance while maintaining low computational requirements and real-time usability.

5.5 Qualitative Analysis

One of the most successful examples involved medical severity detection. When the user uploaded a medical report containing abnormal glucose and hemoglobin levels, the system correctly identified the issue as a high-severity case and routed the request to the report analysis module. The generated response included abnormal findings, possible health risks, medical guidance, and precautionary recommendations. Another important observation was the system's handling of incomplete symptom queries. Inputs such as

“fever and headache” or short fragmented medical phrases were still interpreted correctly. This suggests that the preprocessing and routing pipeline can work even with incomplete user inputs. The main limitation observed during evaluation was citation scoring. In several cases, the generated responses were medically correct but still received low citation scores because the evaluator could not properly match the retrieved medical context against its limited reference data.

CHAPTER 6

DISCUSSIONS

The system performed well on its core healthcare tasks. Medical report analysis and symptom-based queries produced reliable responses with minimal hallucination because the outputs were grounded using structured medical data and retrieved context. The OCR and preprocessing pipeline also handled incomplete symptom descriptions and scanned reports effectively. The evaluation results showed strong performance in severity detection, report analysis, OCR extraction, and response generation accuracy. Average response time remained within acceptable limits for real-time medical assistance.

Some limitations were also identified during evaluation. First, the embedding model was not specifically trained for medical terminology, which reduced retrieval similarity for certain clinical terms. Second, citation accuracy appeared low because the evaluator compared responses only with a limited reference context instead of the full retrieval database. Third, CPU-based local inference slightly increased response latency during complex report analysis. Finally, reports containing rare diseases or unclear scan quality sometimes produced incomplete analysis results.

Future improvements include medical-domain fine-tuned embeddings, advanced OCR for handwritten reports, image-based disease prediction, multilingual medical response generation, and GPU-supported deployment for faster large-scale performance.

CHAPTER 7

CONCLUSION AND FUTURE WORK

MedAgent demonstrates that a lightweight, locally deployable healthcare AI system can provide reliable medical assistance through symptom analysis, medical report interpretation, OCR-based extraction, and retrieval-grounded response generation. By combining OCR processing, structured medical analysis, RAG-based retrieval, and local LLM inference, the system achieved strong overall evaluation performance with accurate severity prediction, reliable report analysis, and low hallucination rates.

The system can run on standard computer hardware without requiring expensive cloud infrastructure or high-end GPU resources, making it suitable for resource-constrained healthcare environments. Future work will focus on medical-domain fine-tuned embeddings, advanced OCR improvement, multilingual medical support, image-based disease analysis, and optimized deployment for faster large-scale healthcare assistance.

CHAPTER 8

REFERENCES

- [1] C. Amadi and A. Ojo, “Building Trustworthy AI in Healthcare,” *IEEE Access*, vol. 14, pp. 1184–1208, 2026. Base Paper Link: [Base Paper – Building Trustworthy AI in Healthcare](#)
- [2] Ashish Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. Paper Link: [Transformer Paper](#)
- [3] Tom B. Brown et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020. Paper Link: [GPT-3 Paper](#)
- [4] Jason Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” arXiv preprint arXiv:2201.11903, 2022. Paper Link: [Chain-of-Thought Paper](#)
- [5] Patrick Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. Paper Link: [RAG Paper](#)
- [6] Hugo Touvron et al., “Llama 2: Open Foundation and Fine-Tuned Chat Models,” arXiv preprint arXiv:2307.09288, 2023. Paper Link: [Llama 2 Paper](#)
- [7] [Groq Documentation](#), “Groq API Documentation,” 2025.
- [8] [LangChain Documentation](#), “LangChain Documentation,” 2025.
- [9] [FAISS Documentation](#), “FAISS: Facebook AI Similarity Search,” 2025.
- [10] [Streamlit Documentation](#), “Streamlit Framework Documentation,” 2025.
- [11] [PyMuPDF Documentation](#), “PyMuPDF Documentation,” 2025.
- [12] [Tesseract OCR Documentation](#), “Tesseract OCR Engine,” 2025.
- [13] [Sentence Transformers Documentation](#), “Sentence-Transformers for Semantic Embeddings,” 2025.
- [14] [Python Documentation](#), “Python Official Documentation,” 2025.